



Adaptación de Skills IA al Ecosistema BCV/BACC

Presentación Ejecutiva — Resumen

Proyecto: skills de agente IA para los microservicios backend de Apertura de Cuentas Comerciales (BACC)

Ecosistema: BCV/BACC · Interbank · Entregable: NTT DATA

Cronograma: 13 ago → 09 oct 2026 · Sprint 2 en curso (4 sprints de 2 semanas)

Alcance del Proyecto

7

microservicios backend

3

skills del pipeline

~85 %

menos tokens por HU

8 sem.

13 ago → 09 oct 2026

¿Qué es este proyecto?

El ecosistema BCV/BACC de Interbank opera sobre 7 microservicios backend Java/Spring Boot. El proyecto construye un pipeline de 3 skills de agente que convierte una Historia de Usuario (HU) de negocio en una Historia Técnica (DHU) y, opcionalmente, en código — usando graphify para reducir el consumo de tokens en ~85 %.

Metodología: SDD + BMAD

El proyecto sigue Spec-Driven Development (SDD) y BMAD:

SDD

Framework de desarrollo guiado por especificaciones. Antes de escribir código se produce y aprueba una spec (la DHU) con criterios de aceptación y mapa técnico — bcv-hu-implementer no genera código si la DHU tiene gaps sin resolver.

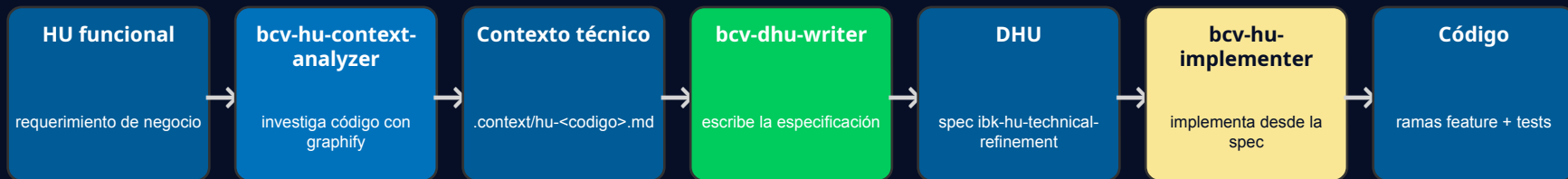
BMAD

Framework de desarrollo ágil con IA. Es el ciclo interno que cada skill sigue al construir algo: Understand (entender la tarea y el código), Design (diseñar el cambio), Build (generarlo) y Validate (verificarlo).

Fase SDD	Acción del flujo BCV
Intención de entrada	HU funcional de negocio
Research-Driven Context	bcv-hu-context-analyzer investiga el código real con graphify → .context/hu-<codigo>.md
Especificar formalmente	bcv-dhu-writer escribe la DHU (criterios de aceptación, endpoints, mapa técnico)
Implementar desde la spec	bcv-hu-implementer aplica la DHU en ramas feature (dry-run / apply)
Verificar / feedback	Linter + tests, revisión humana y gotchas que vuelven al contexto

Skills que se Identificaron

Se identificó y construyó un pipeline de 3 skills principales, uno por cada paso del flujo SDD:



Primera entrega: un plan de tareas validado

bcv-hu-context-analyzer (research con graphify) y bcv-dhu-writer (especificación) ejecutan la primera parte del pipeline y entregan un plan de tareas (la DHU) validado. Desde ahí, bcv-hu-implementer aplica el plan y genera el código, iterando con el feedback del equipo dev.

Resultados (hitos medidos)

Los dos primeros ejercicios con el equipo dev del banco cuantificaron el ahorro frente a la forma tradicional:

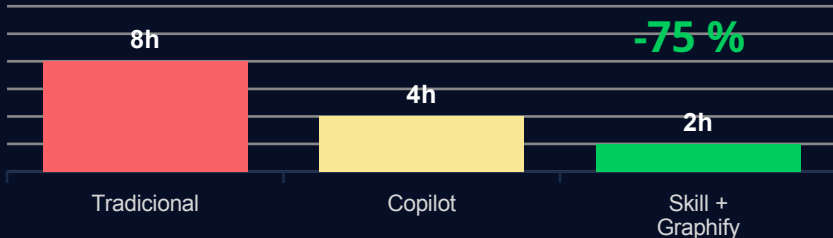
Usuario	HU / EVT	Compl.	Tradic. (h)	Copilot (h)	Skill+Graphify (h)	Ahorro
Fernando Camargo	Identificación de canal OneAPP en trama Postmortem	Baja	8	4	2	75 %
Lionel Gonzales	[EVT] Arquitectura cluster-to-cluster – Paquete 1	Media	48	40	10	79 %

Fernando Camargo

HU · complejidad baja

75 %

ahorro

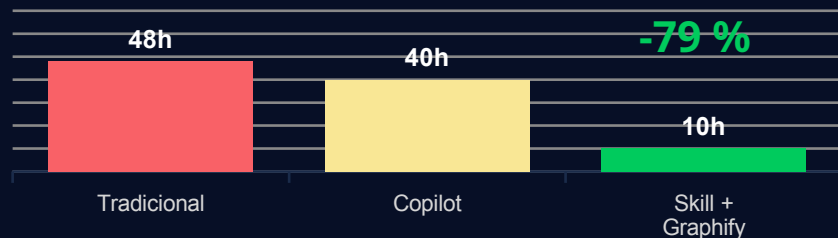


Lionel Gonzales

EVT · complejidad media

79 %

ahorro



En el caso de mayor complejidad (Lionel Gonzales), pasar de 48 h a 10 h equivale a un ahorro de ~79 % (~4 días de implementación).

Roadmap

Completada

En curso

Propuesto



SPRINT 1		SPRINT 2		SPRINT 3		SPRINT 4		
13–26 ago		27 ago–09 sep		10–23 sep		24 sep–09 oct		
SEM 1	SEM 2	SEM 3	SEM 4	SEM 5	SEM 6	SEM 7	SEM 8	SEM C
13–19 ago	20–26 ago	27 ago–02 sep	03–09 sep	10–16 sep	17–23 sep	24–30 sep	01–07 oct	08–09 oct
Consolidación de repos + entorno	Adopción de graphify	Iteración de los skills del pipeline	Consolidar feedback de la ejecución EVT	Skills adicionales: bcv-hexagonal-architecture y bcv-openapi-design	Consolidar el pipeline completo	Validación con equipo dev del banco (entorno propio)	Estabilización final de skills	Cierre final de entregables
Arranque de los 3 skills del pipeline	Ejecución de HU con equipo dev (primer ejercicio real)	Ejecución EVT con equipo dev (context-analyzer → DHU → código)	Iterar skills del pipeline con código real	Sumar bcv-spring-data-jpa-sql-server (persistencia SQL Server)	Ejecutar bcv-hu-implementer (dry-run → apply)	Ciclo: ajustar → regenerar → revalidar	Documentación, onboarding y demo al cliente	Traspaso a cargo de Oscar

Próximos Pasos

1

Skills adicionales: **bcv-hexagonal-architecture** y **bcv-openapi-design**

Complementan el pipeline: **hexagonal-architecture** define el patrón de todo el código, y **openapi-design** define lo que se expone y valida la HU contra su contrato. **bcv-spring-data-jpa-sql-server** (persistencia SQL Server, usada por casi toda HU) se suma como tercer complemento.

2

Iterar los skills en las próximas HUs

Ampliar su cobertura con casos de uso adicionales del banco, sobre Historias de Usuario reales.

3

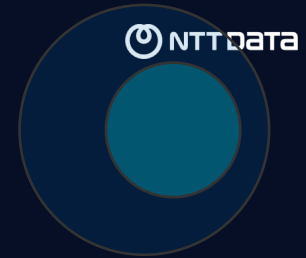
Oscar queda a cargo del proyecto

Tras el traspaso del Arquitecto IA (10 sep): soporte continuo y afinación de los skills.

4

Refinar los skills

Con más código real y el feedback continuo del equipo dev del banco.



Skills IA para el Ecosistema BCV/BACC

Pipeline HU → DHU → código · 3 skills · ~85 % menos tokens

Próximos pasos: iterar y refinar los skills en las próximas HUs, con Oscar a cargo del proyecto